CS 498 · Machine Learning Systems

# Adaptive KV Cache

## Which tokens should an LLM remember?

H2O · StreamingLLM · Memory / accuracy tradeoffs

Fan Lai

University of Illinois Urbana-Champaign | Fall 2026

Thanks to Minjia Zhang, Arvind Krishnamurthy, and colleagues

# Course reminders

## Mid-semester project report

Due Wed, Nov 4 · 6:00 PM CST

- Four pages: problem, quantitative motivation, related work, approach, and evaluation.
- Please show measurable progress, not just the original proposal.

## Assignment 2

Due Mon, Nov 9

Attention, LoRA, PagedAttention, and MoE exercises

Panel discussion connection: SGLang and KV-cache reuse

# Today’s lecture

From page allocation to deciding which tokens are worth keeping

| | | |
|---|---|---|
| 01 | **Motivation** | Why does KV memory keep growing after PagedAttention? |
| 02 | **H2O** | Can attention history identify useful KV tokens? |
| 03 | **StreamingLLM** | Why retain initial sinks along with a recent window? |
| 04 | **Compare & apply** | How should we evaluate and choose a KV policy? |

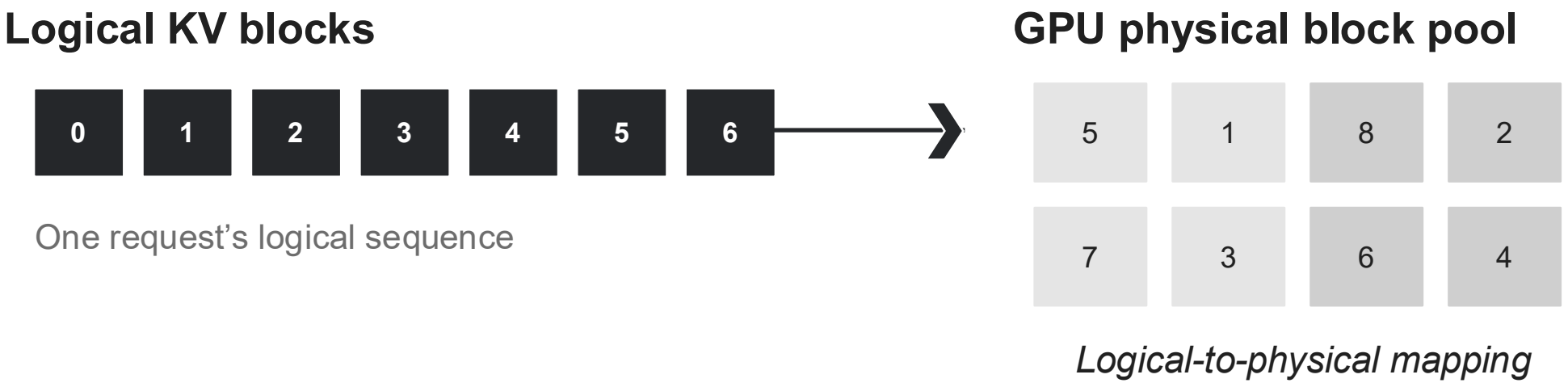
Panel connection: SGLang reuses KV across requests; today we study what to retain within a request.

# Learning Objectives

By the end of class, you should be able to…

| | |
|---|---|
| **Explain** | why paging reduces fragmentation but cannot bound KV memory. |
| **Apply** | H2O and StreamingLLM to a fixed cache budget. |
| **Evaluate** | memory savings against accuracy, latency, and SLO goodput. |

# Recall: PagedAttention improves memory allocation




**01** **Avoid external fragmentation**

Physical blocks need not be contiguous.

**02** **Limit internal fragmentation**

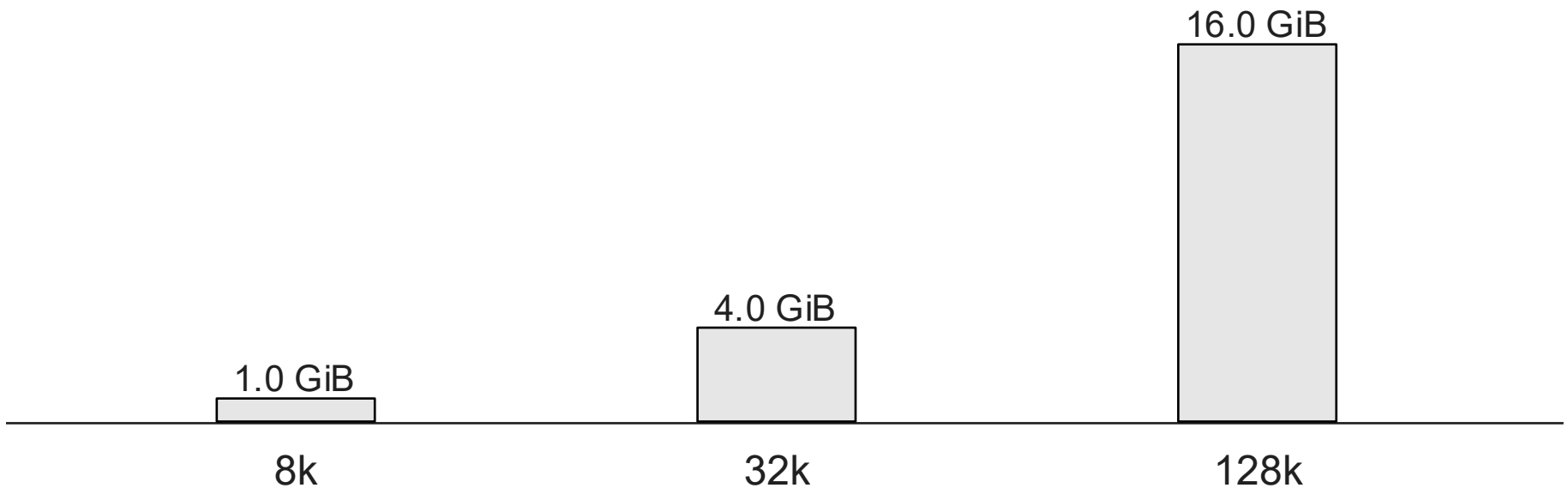
Waste per sequence is smaller than one block.

Background: Kwon et al., "Efficient Memory Management for Large Language Model Serving with PagedAttention," SOSP 2023.

# PagedAttention does not reduce the KV data itself

**KV bytes per request ≈ 2 × $L_{(layers)}$ × $H_{(\#\ of\ grouped\ head)}$ × $D_{(dimension)}$ × $T_{(length)}$ × bytes/value**

Illustrative model configuration: 32 layers, 8 KV heads, head dim 128, BF16




**Even perfect page packing cannot prevent linear growth in T** (sequence length)

Illustrative arithmetic (per request, KV cache only); actual memory depends on architecture, precision, and implementation.

# Why not simply keep a smaller KV cache?

Removing KV pairs saves memory yet may change future predictions.

**01 Capacity**

A bounded number of KV tokens per request.

**02 Model quality**

Do not discard history that matters later.

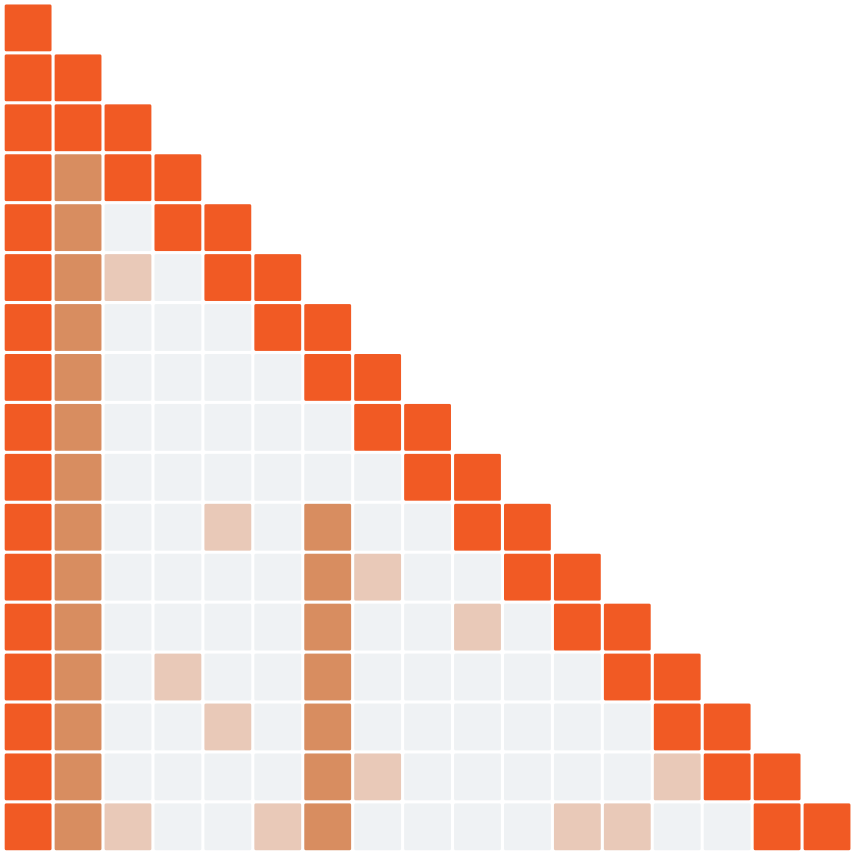
**03 Policy overhead**

Eviction should not cost more than it saves.

**Can attention provide an inexpensive retention signal?**

# Attention is often sparse

## Illustrative causal attention matrix



Schematic, not an experimental attention matrix.

## Key position

- Many scores are near zero.
- A small subset receives large attention.
- Recent tokens can matter too.

**But sparse today does not mean useless tomorrow.**

# What makes a token worth retaining?

Consider token i, currently stored in the KV cache.

It may be...

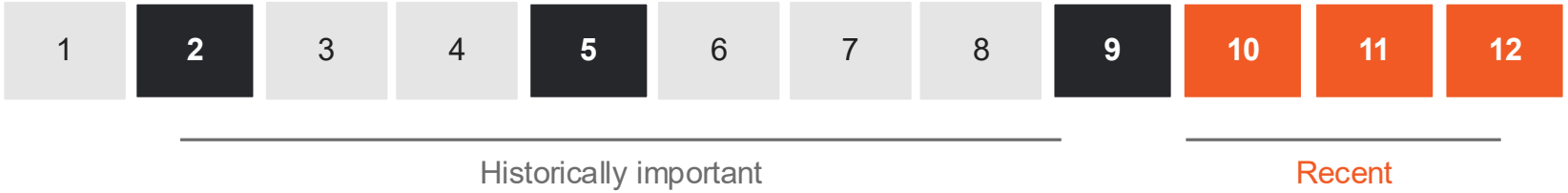
- Useful because many later tokens attended to it.
- Useful because it was just generated.
- Useful only for a future question we have not seen.

**QUICK PREDICTION: Which retention signal would you trust?**

# Two approaches to a bounded KV cache

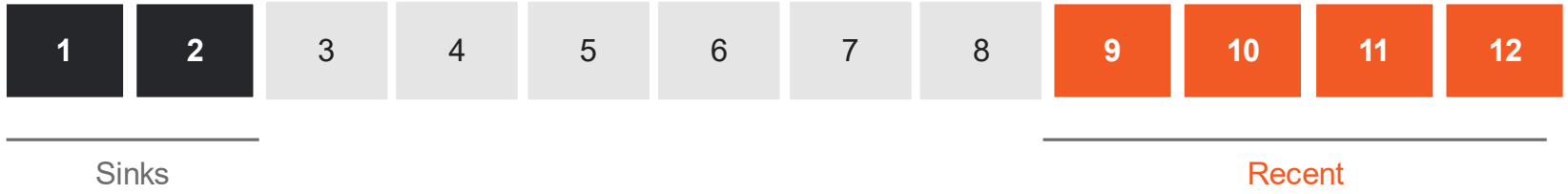
## H2O

Keep high-attention history and recent tokens




## StreamingLLM

Keep initial attention sinks and recent tokens

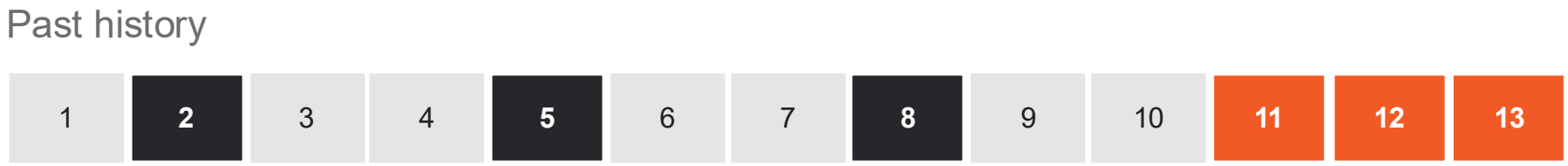



Orange = recent-window KV; black = heavy hitters / sinks; gray = evicted KV (illustrative).

# H2O: attention as a retention signal

**Key idea: not all past tokens matter equally.**

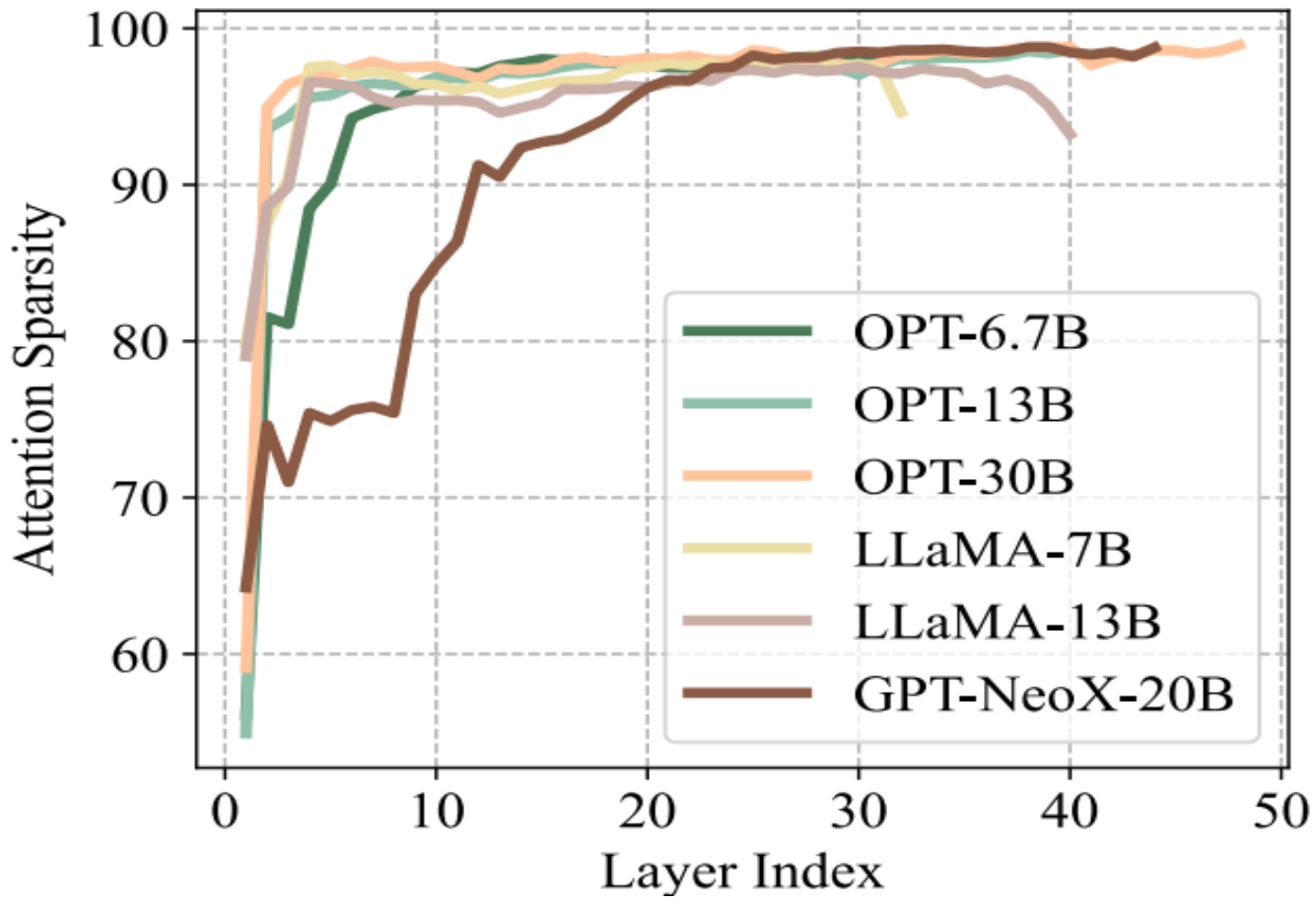
| | |
|---|---|
| **Observe** | Which earlier tokens receive attention? |
| **Accumulate** | Estimate importance across decode steps. |
| **Retain** | Reserve slots for heavy hitters and recency. |




What we keep depends on observed attention—not token age alone.

H2O: Zhang et al., “Heavy-Hitter Oracle,” NeurIPS 2023. Selection shown schematically.

# H2O observation 1: attention is sparse




## Attention sparsity

The paper reports >95% sparse scores in most layers under its threshold.

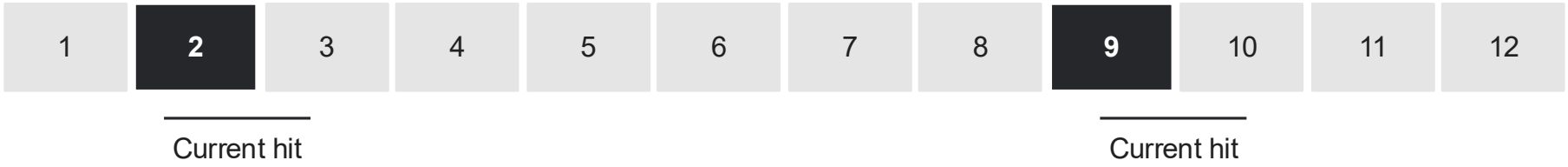
## Takeaways

Full access to all past KV may be unnecessary for each step.

Source: H2O, NeurIPS 2023, attention-sparsity result; figure reused from supplied lecture.

# Challenge 1: Sparse attention ≠ an eviction policy

Current token: attends to positions 2 and 9




Next token: what if it needs position 5?

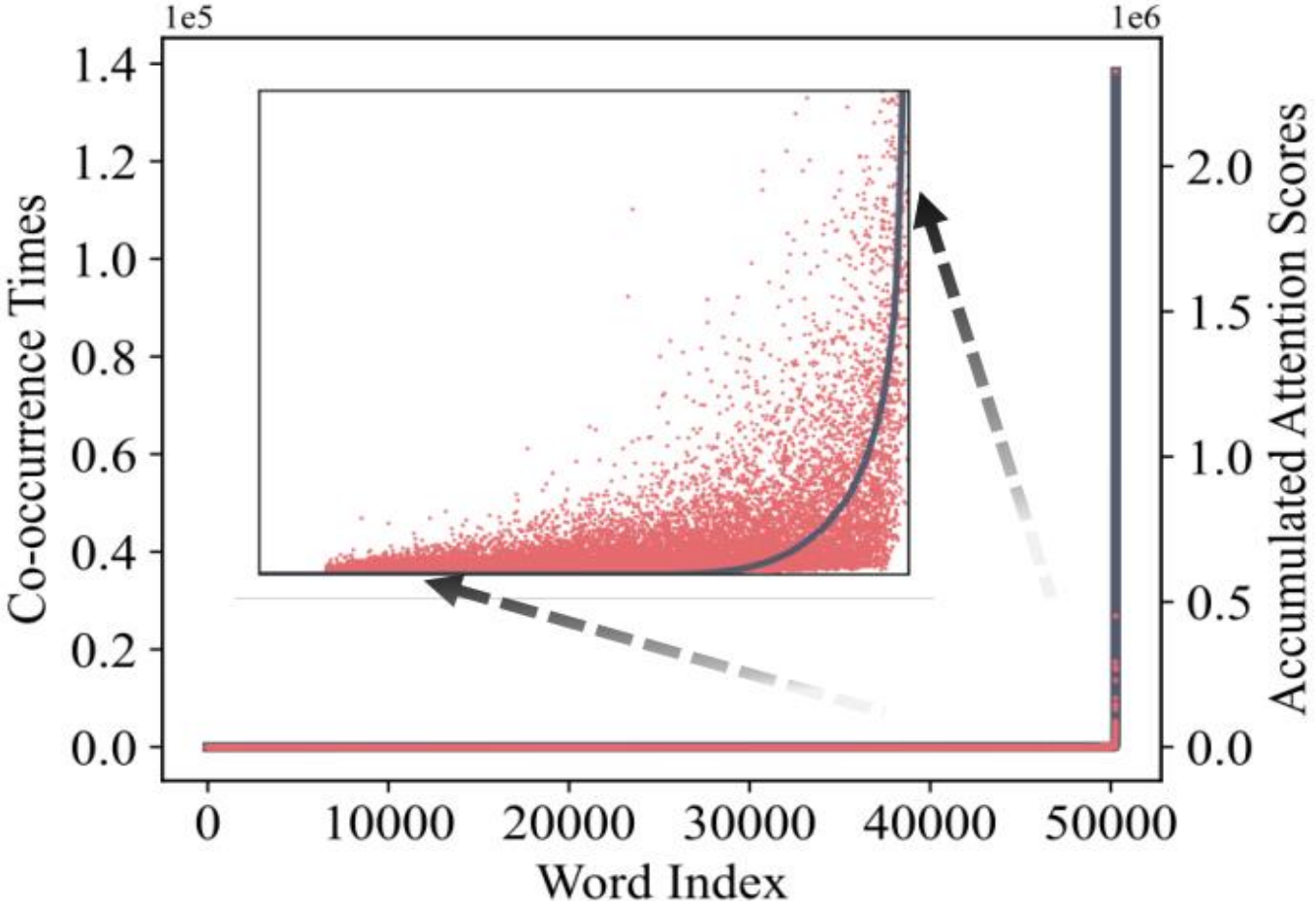



**We do not observe future attention before generating future tokens.**

**Need a retention signal that is cheap and robust over time.**

Illustrative sequences; not reproduced from the paper.

# H2O observation 2: token importance is skewed



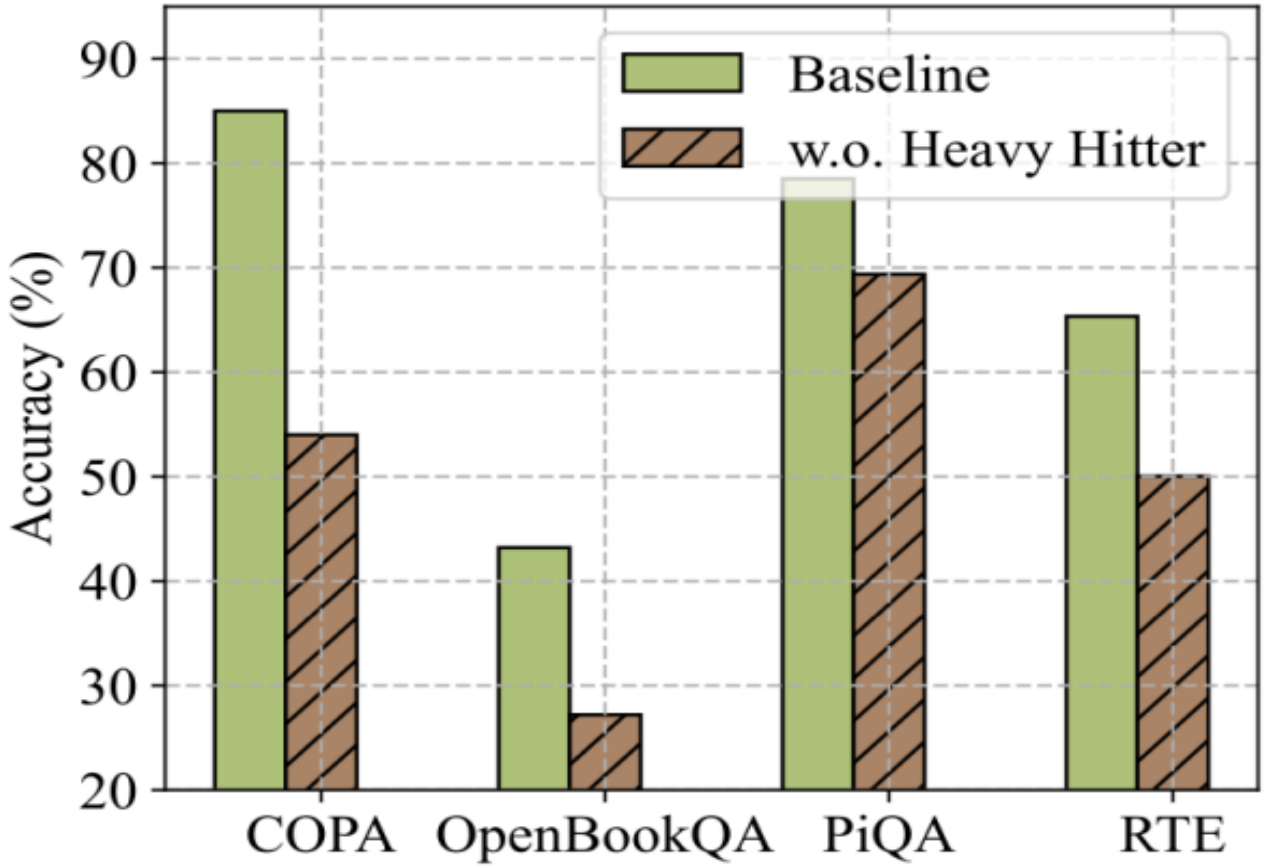

## Heavy hitters

- Accumulated attention follows a highly unequal distribution.
- A small set of tokens repeatedly attracts attention.

**Candidate signal: accumulated attention.**

Source: H2O, NeurIPS 2023, accumulated attention / co-occurrence figure; supplied lecture.

# H2O observation 3: heavy hitters affect accuracy




**Evicting the wrong KV hurts**

Removing heavy hitters reduces accuracy across evaluated reasoning tasks.

Source: H2O, NeurIPS 2023, heavy-hitter ablation; supplied lecture.

# How H2O scores tokens

Accumulate the attention each stored token receives over time.

**score(i, t+1) = score(i, t) + attention(t+1 → i)**

The new attention contribution is added to the previous score.

**Example: score update at the next decoding step**

| Token 1 | Token 2 | Token 3 | Token 4 | Token 5 |
|---|---|---|---|---|
| 0.8 | 3.1 | 0.7 | 2.4 | 0.5 |
| +0.1 | +0.6 | +0.0 | +0.2 | +0.1 |
| 0.9 | **3.7** | 0.7 | **2.6** | 0.6 |

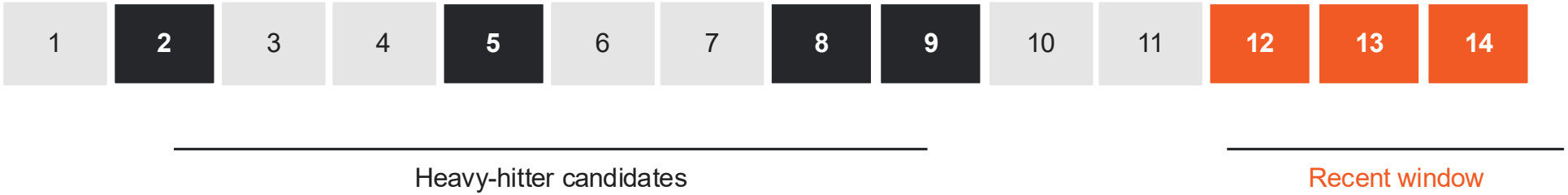
**Takeaways** Cheap online scores approximate future value. Recency still needs protection.

Conceptual illustration. A[j,i] is attention from generated token j to stored token i.

# H2O keeps heavy hitters and recent tokens

At a fixed budget, reserve space for two kinds of history.

Past tokens → next decode step




**Historical importance**

Retain useful older context

**Recency**

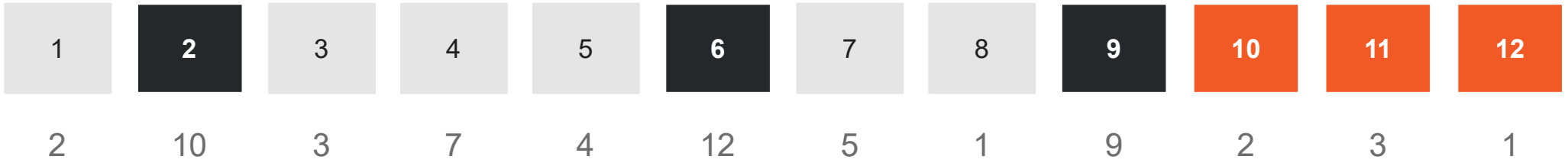
Protect newly created context

Selection shown schematically; token choices vary by workload, head/layer, and budget.

# Example: allocate six KV slots

12 existing tokens · 6 slots total · 3 reserved for recent tokens

**Accumulated scores**



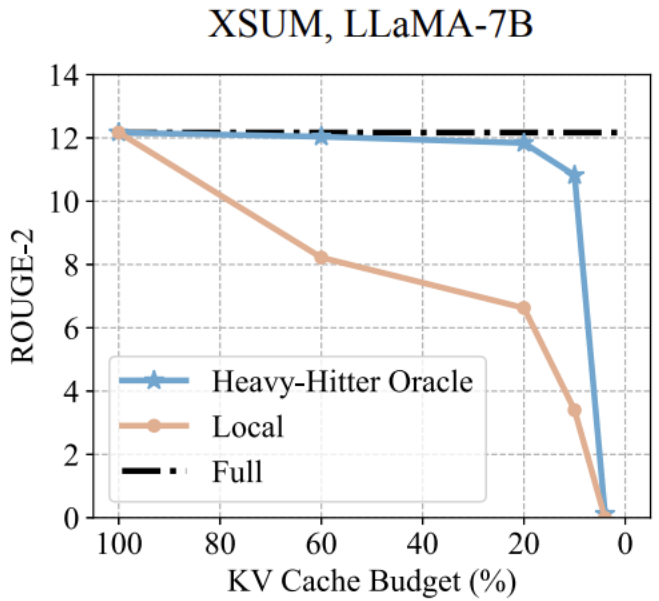

**Chosen:** Heavy hitters = {2, 6, 9}

Recent = {10, 11, 12}

**Token 4 scores relatively well, but loses to 2 / 6 / 9.**

Toy example. H2O is a head/layer-aware online policy, not this single global table.

# H2O: smaller caches can preserve quality

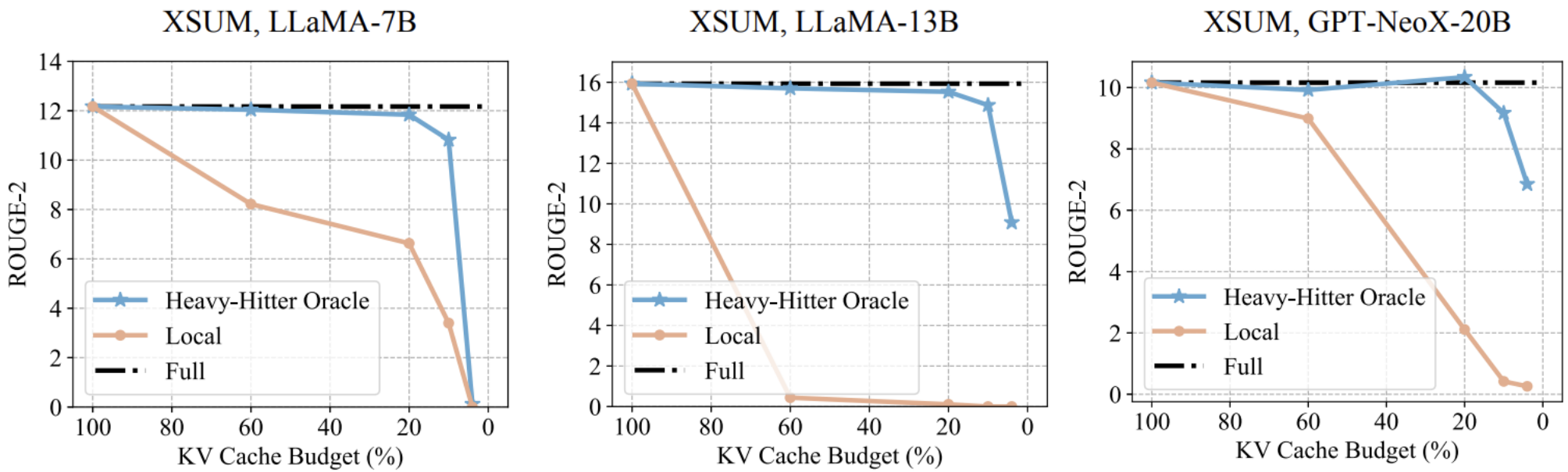



ROUGE / task accuracy vs. retained KV budget

**Keeping high-importance tokens beats naive local eviction at constrained budgets.**

Source: H2O, NeurIPS 2023, comparative performance curves; supplied lecture.

# H2O: smaller caches can preserve quality




ROUGE / task accuracy vs. retained KV budget

**Keeping high-importance tokens beats naive local eviction at constrained budgets.**

Source: H2O, NeurIPS 2023, comparative performance curves; supplied lecture.

# Where can H2O struggle?

**01** **Future relevance is unknown**

A low-scoring token may become critical after a topic change.

**02** **Eviction is often irreversible**

Without recomputation or offloading, removed KV is gone.

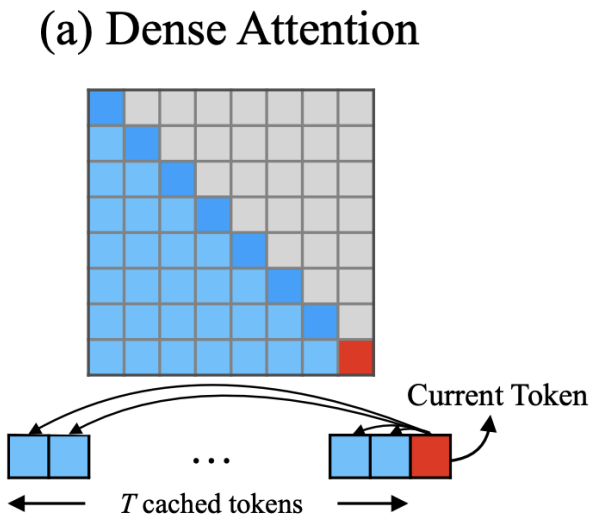
**03** **Scores and eviction have cost**

Attention tracking, ranking, and memory movement compete with decode.

**How about others?**

# StreamingLLM: why does a sliding window fail?

## Alternative attention strategies




$O(T^2)$✗ **PPL: 5641**✗

Has poor efficiency and performance on long text.

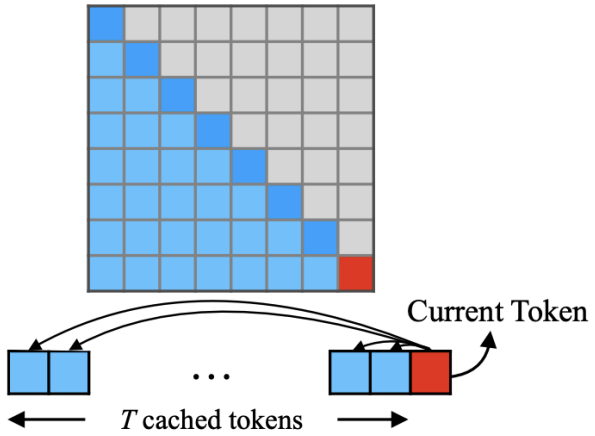
Lower perplexity (PPL) indicates better response quality.

Source: Xiao et al., “Efficient Streaming Language Models with Attention Sinks,” ICLR 2024; supplied lecture.

# StreamingLLM: why does a sliding window fail?

## Alternative attention strategies

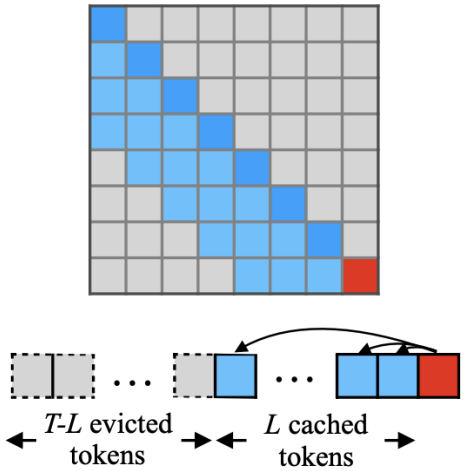
(a) Dense Attention




$O(T^2)$✗ **PPL: 5641**✗

Has poor efficiency and performance on long text.

(b) Window Attention




$O(TL)$✓ **PPL: 5158**✗

Breaks when initial tokens are evicted.

Lower perplexity (PPL) indicates better response quality.

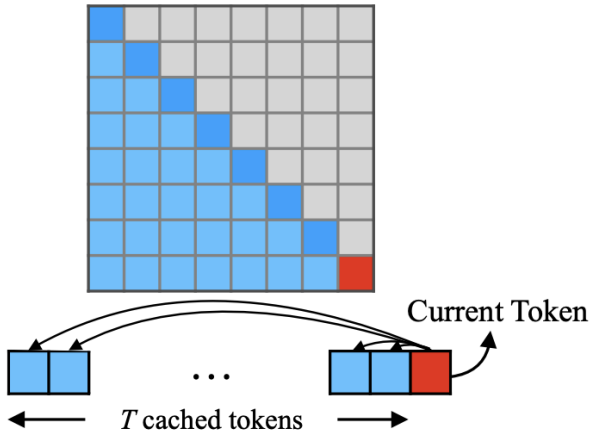
(better PPL due to mitigating lost-in-the-middle issue)

Source: Xiao et al., "Efficient Streaming Language Models with Attention Sinks," ICLR 2024; supplied lecture.

# StreamingLLM: why does a sliding window fail?

## Alternative attention strategies

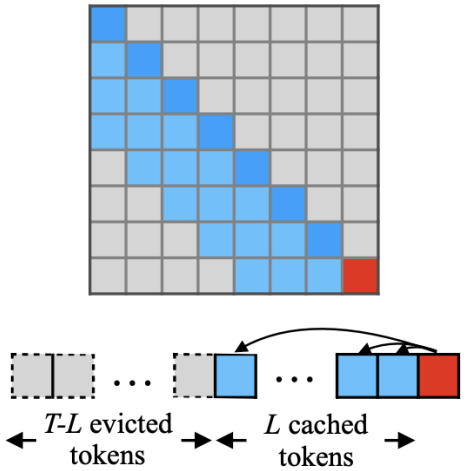
(a) Dense Attention




$O(T^2)$✗ **PPL:** 5641✗

Has poor efficiency and performance on long text.

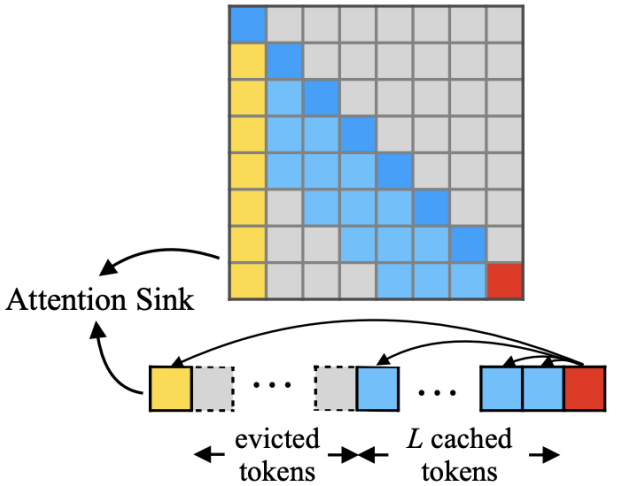
(b) Window Attention




$O(TL)$✓ **PPL:** 5158✗

Breaks when initial tokens are evicted.

**StreamingLLM (ours)**




$O(TL)$✓ **PPL:** 5.40✓

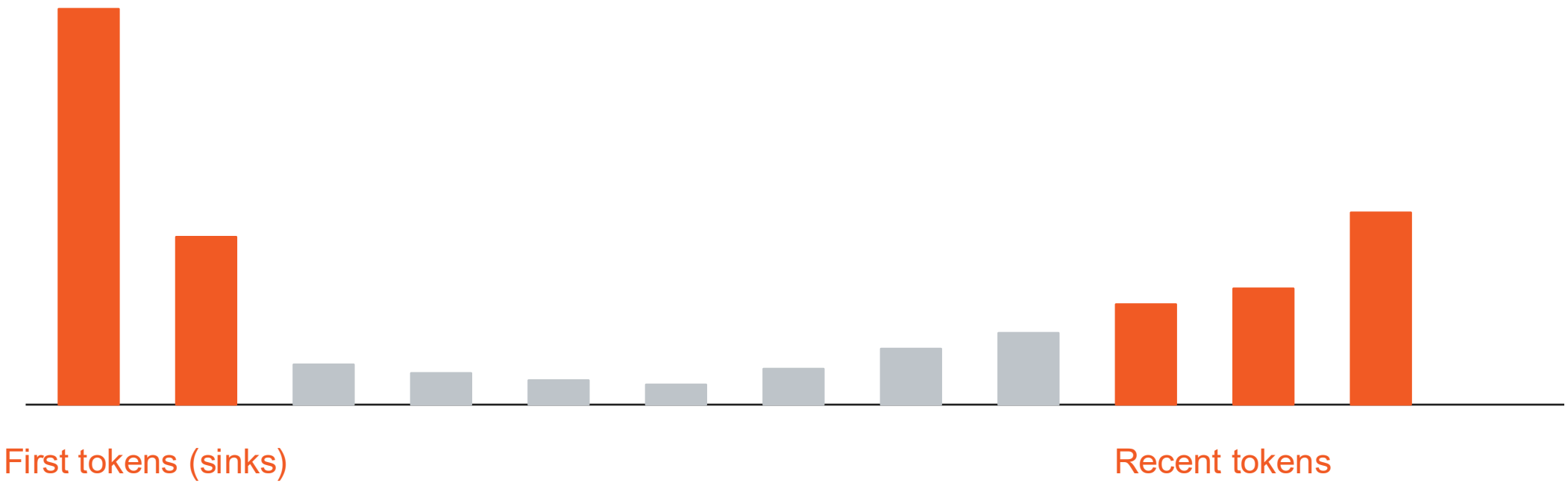
Can perform efficient and stable language modeling on long texts.

Window-only eviction can destabilize generation even with recent tokens present.

Source: Xiao et al., "Efficient Streaming Language Models with Attention Sinks," ICLR 2024; supplied lecture.

# Attention sinks: a few early positions attract attention

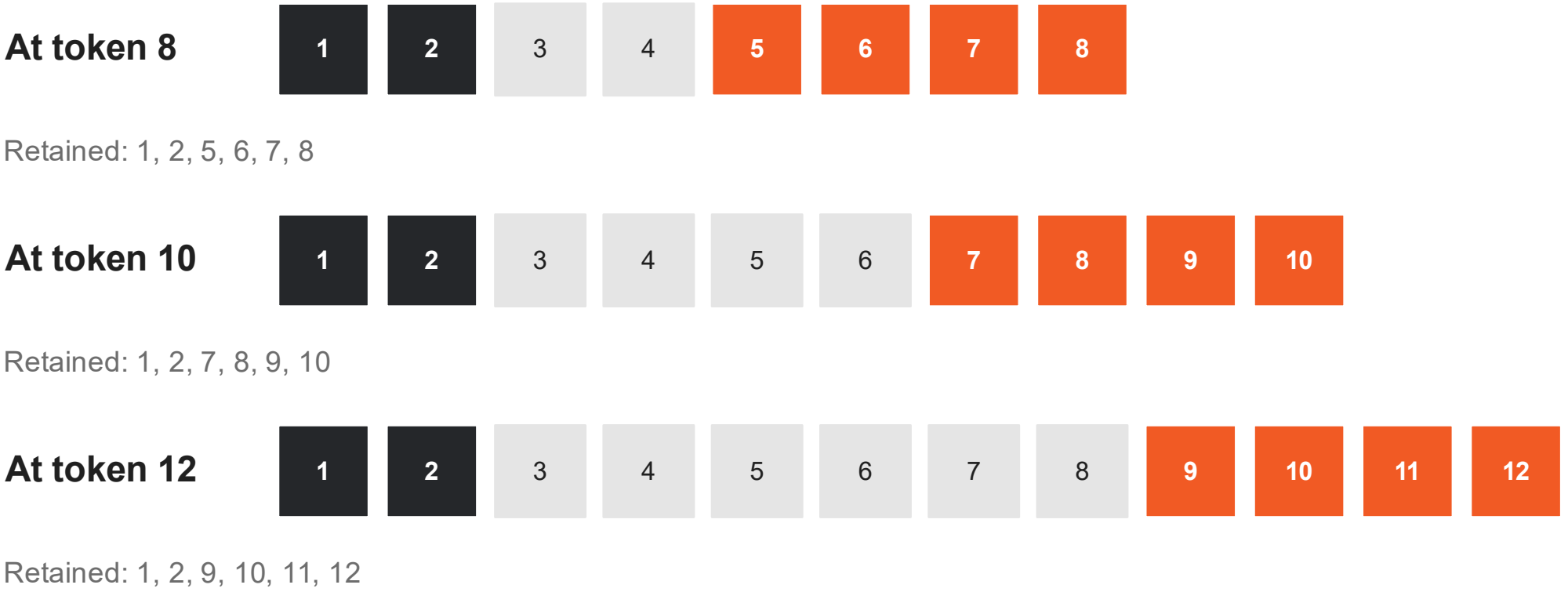
**Schematic attention mass for the next token**




**Attention sink ≠ a token is necessarily semantically relevant.**

Schematic phenomenon; actual attention depends on layer, head, model, and sequence.

# StreamingLLM: fixed sinks + a rolling window




**Bounded KV memory even as the generated stream grows.**

Illustrative 2 sink tokens + a 4-token recent window.

# Why are attention sinks useful?

**01** **Softmax allocates probability mass**

Some attention heads learn to assign excess mass to early positions.

**02** **A pure window deletes those positions**

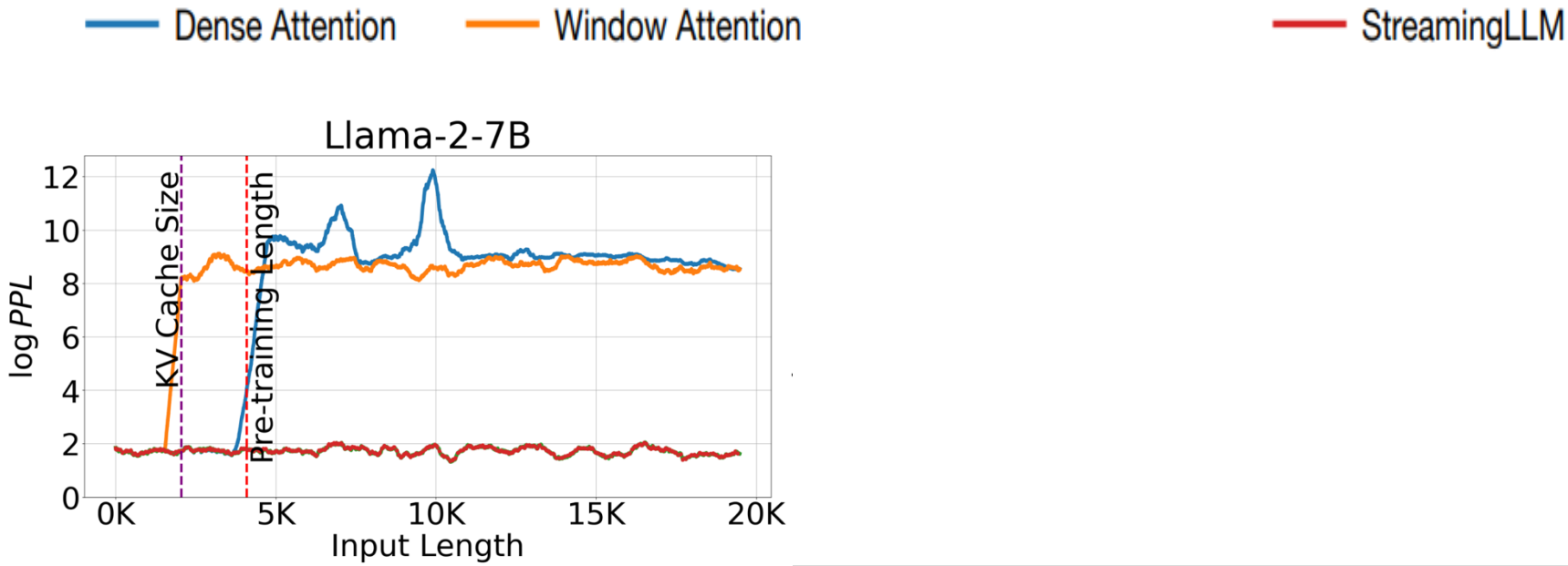
The resulting attention pattern differs from training-time behavior.

**03** **Preserving a few sinks stabilizes decoding**

Keep the first tokens while moving the recent window forward.

Xiao et al., StreamingLLM (ICLR 2024), analysis of attention-sink behavior.

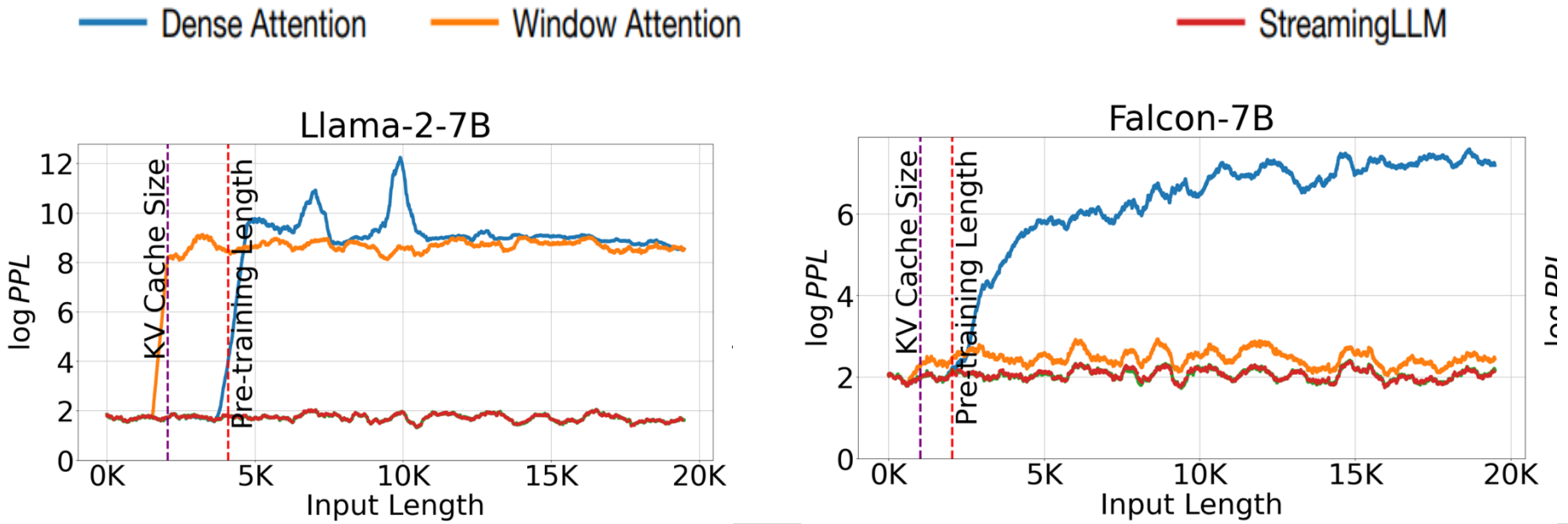
# StreamingLLM stabilizes long-stream perplexity




**Window-only attention can become unstable after sink tokens disappear.**

Source: Xiao et al., StreamingLLM (ICLR 2024), perplexity across long inputs; supplied lecture. Lower is better.

# StreamingLLM stabilizes long-stream perplexity




**Window-only attention can become unstable after sink tokens disappear.**

Source: Xiao et al., StreamingLLM (ICLR 2024), perplexity across long inputs; supplied lecture. Lower is better.

# One question, two different policies

A 12-token history with six KV slots

**H2O**




Important history + last 3 tokens

**Streaming LLM**




First 2 sink tokens + last 4 tokens

**Different goals → different retained history.**

Illustrative six-slot choices; H2O selection depends on observed attention.

# Takeaways: Compare full KV, H2O, and StreamingLLM

| Property | Full / paged KV | H2O | StreamingLLM |
|---|---|---|---|
| Retained history | All tokens | Heavy + recent | Sink + recent |
| Memory grows? | O(T) | Bounded budget | Fixed window |
| Best strength | Exact history | Adaptive importance | Long stream stability |
| Primary risk | Memory pressure | Future relevance | Old facts lost |

**Policy choice depends on task, context dynamics, and serving constraints.**

# Exercise: which six tokens would you keep?

At token 12, your KV budget is six slots.

| | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Accumulated attention scores | 2 | 10 | 3 | 7 | 4 | 12 | 5 | 1 | 9 | 2 | 3 | 1 |

1. H2O: 3 heavy + 3 recent?
2. StreamingLLM: 2 sinks + 4 recent?

# Answer: three policies retain different histories

**Full KV**



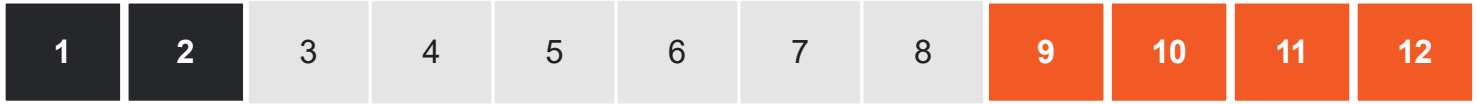

All 12 retained

**H2O**




2, 6, 9 + 10–12

**StreamingLLM**




1, 2 + 9–12

Full KV example is unconstrained; H2O and StreamingLLM are constrained to six slots.

# Open discussions

## Dynamic KV budget

How much retained history is worth its memory under tight SLO slack?

## Tiered KV management

When to evict, offload, prefetch, or recompute?

## Agent context

Can future tool calls tell us which earlier tokens will matter?

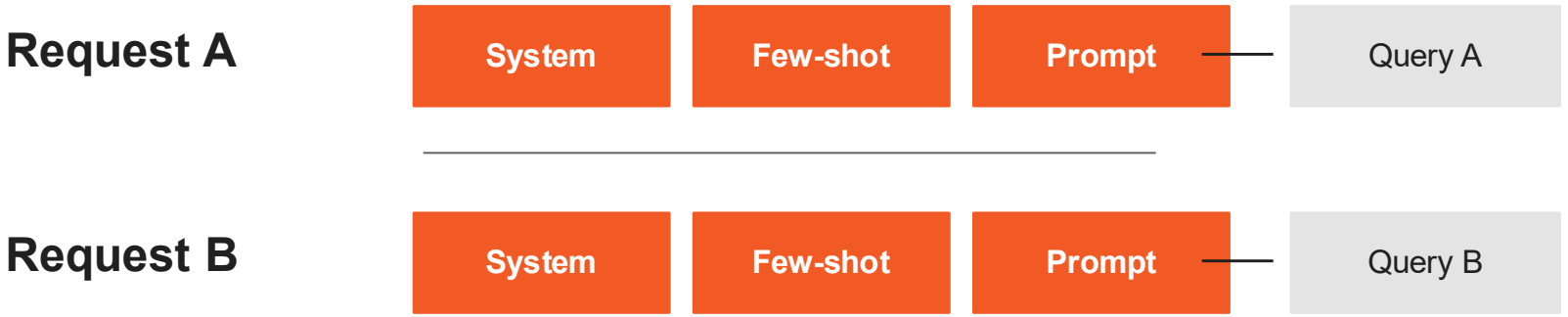
## Joint scheduling

Do KV residency and batching need one coordinated controller?

Research directions extend beyond the original H2O / StreamingLLM proposals.

# Panel Preview: SGLang and KV-cache reuse

**Key idea: Shared prefix → reuse prefilled KV using RadixAttention**




**Discuss:** Cache-aware scheduling, eviction under memory pressure, and where H2O differs.

Zheng et al., "SGLang: Efficient Execution of Structured Language Model Programs," NeurIPS 2024. Schematic.